



Australian
National
University

LoRA

Efficiently adapt a trained language model to a new domain or style without retraining the entire model from scratch.

You will need

- a completed bigram model from an earlier lesson
- pen, pencil and grid paper
- some new text in a different style or domain

Your goal

To create a lightweight “adaptation layer” that modifies your existing model’s behaviour for a new domain. **Stretch goal:** combine the base model and LoRA layer with different mixing ratios.

Key idea

Low-Rank Adaptation (LoRA) allows you to specialise a language model by adding small adjustments rather than retraining everything. The LoRA layer is typically much smaller than the base model because you only track the *changes* from the base model, not the full weights.



Algorithm

1. choose an existing bigram model as the “base model”
2. train a LoRA layer:
 - start with a new grid (same columns as the base model)
 - process your new domain-specific text using the same algorithm as *Basic Training*, but only include rows for words that appear in your new text
3. apply the adaptation:
 - as per *Basic Generation*, but add the counts from both grids (if current word is in the LoRA grid)
 - optionally scale the LoRA values up or down to control adaptation strength

Example

Base model trained on general text:

	saw	they	we	the	a	red
saw		2		4	2	1
they	1			2	1	
we				3		
the			1			
a						2
red		1				

LoRA layer trained on “I saw a red cat. I saw the red dog.” (smaller — only 1 row):

	saw	they	we	the	a	red
saw				1	1	2

Combined model (add counts):

	saw	they	we	the	a	red
saw		2		5	3	3
they	1			2	1	
we				3		
the			1			
a						2
red		1				

- **saw** row:
 - $[-, 2, -, 4, 2, 1]$ (base)
 - $[-, -, -, 1, 1, 2]$ (LoRA)
 - $[-, 2, -, 5, 3, 3]$ (base + LoRA)
- **red** now equally likely as **the** after **saw**
- other rows: base + zero = unchanged
- LoRA is smaller: only 1 row vs 6 in base model